Programming Assignment 3 (PA3)

Magical Demon Hunters Game
User-Defined Types, Encapsulation, Operator Overloading, Interacting Objects
Out: March 11, 2026 -- DUE: April 7, 2026
EC327 Introduction to Software Engineering – Fall 2026

Total: 200 points

- *You may use any development environment you wish, as long as it is ANSI C++ 11 compatible. Please make sure your code compiles and runs properly under the Linux/Unix environment on the PHO 305/307 (or eng-grid) machines before submitting.*
- *PAs may be submitted up to a week late at the cost of a **30% fixed penalty** (e.g., submitting a day late and a week late is equivalent). It is in your best interest to complete as many checkpoints as possible before the deadline. If you have missing implementations in your original submission, you may complete and submit the missing solutions during the following week. Any submissions after the deadline will be subject to the 30% penalty. No credit will be given to solutions submitted after the 1-week late submission period following the deadline.*
- *Follow the assignment submission guidelines in this document or you will lose points.*

Submission Format (Must Read)

- Use the **exact** file names specified in each problem for your solutions.
- Push your changes to the repository generated by GitHub Classroom.
- Complete submissions should have **22 files.**
- Please do **NOT** submit *.exe and *.o or any other files that are not required by the problem.
- **Code must compile in order to be graded. Otherwise, this is an automatic zero.**
- Comment your code (good practice!). We **may** use your comments when grading.

Coding Style (reminder from PA2) As you become an experienced programmer, you will start to realize the importance of good programming style. There are many coding "guidelines" out there and it is important that you begin to adopt your own. Naming conventions will help you recognize variable names, functions, constants, classes etc. Similarly, there are many ways to elegantly format your code so that it is easy to read. All of these issues do not affect compilation and hence are easy to overlook at this stage in your development as a programmer. However, your ability (or inability) to create clean, readable code could be the difference in your career when you leave college.

Good reference: https://google.github.io/styleguide/cppguide.html If you find any other good references, feel free to post the link on Piazza!

Class Specifications

Each class and its members are described below by listing its name, the prototypes for the member functions, and the names and types of the member variables. You **MUST** use the prototypes, types, and names in your program as specified here, in the same upper/lower case. Function argument names are allowed to be different. **Failure to follow these specifications will result in lost points.**

Checkpoint 3

Mage

(30 Points)

This class inherits from `GameObject`. It will represent objects that move around and can be commanded to do things. It is responsible for the data and functions for moving around, recovering mana and battling in DemonHideouts.

State Machine Enums:

The following enumerated type should be declared in the `Mage.h`

```
enum MageStates {
    STOPPED             = 0,
    MOVING              = 1,
    KNOCKED_OUT         = 2,
    AT_SPIRE            = 3,
    IN_HIDEOUT          = 4,
    MOVING_TO_SPIRE     = 5,
    MOVING_TO_HIDEOUT   = 6,
    BATTLING_IN_HIDEOUT = 7,
    RECOVERING_MANA     = 8
};
```

Public members:

- `Mage();`
  - It initializes the speed to 5 and outputs a message: **"Mage default constructed."**
- `Mage(char in_code);`
  - It initializes the speed to 5 and outputs a message: **"Mage constructed."**

- Also sets initial values as follows:
  * State: STOPPED
  * Display_code: in_code

- `Mage(string in_name, int in_id, char in_code, unsigned int in_speed, Point2D in_loc);`
  - Initializes the speed to `in_speed` and sets name to `in_name`.
  - It outputs a message: **"Mage constructed"**.

- `void StartMoving(Point2D dest);`
  - Tells the Mage to start moving.
  - Calls the `setup_destination()` function.
  - Sets the state to `MOVING`
  - If this Mage is already at the destination print "**(display_code)(id): I'm already there. See?"**
  - If this Mage is knocked out print **"(display_code)(id): I am knocked out. I may move but you cannot see me."**
  - Otherwise prints **"(display_code)(id): On my way."**

- `void StartMovingToHideout(DemonHideout* hideout);`
  - Tells the Mage to start moving to a DemonHideout.
  - Calls the `SetupDestination()` function with DemonHideout's location as the destination.
  - Sets the state to `MOVING_TO_HIDEOUT`
  - If this Mage is knocked out print **"(display_code)(id): I am knocked out so I can't move to hideout..."**
  - Prints the message **"(display_code)(id): on my way to hideout (hideout id)."**
  - If this Mage is already there print **"(display_code)(id): I am already at the Demon Hideout!"**

- `void StartMovingToSpire(ManaSpire* spire);`
  - Tells the Mage to start moving to a ManaSpire.
  - Calls the `SetupDestination()` function with ManaSpire's location as the destination.
  - Sets the state to `MOVING\_TO\_SPIRE`
  - If this Mage's has lost all mana print **"(display_code)(id): I am knocked out so I should have gone to the spire.."**
  - Prints the message **"(display_code)(id): On my way to Spire (spire id)"**
  - If Mage is already there print **"(display_code)(id): I am already at the Spire!"**

- `void StartBattling(unsigned int num_battles);`

- Tells the Mage to start battling (`num_battles`) in a DemonHideout.
- Sets the state to `BATTLING_IN_HIDEOUT` and prints the message **"(display_code): Started to battle at the DemonHideout (hideout id) with (number of battles) battles"** unless the following conditions are true:
  * This Mage is out of mana print **"(display_code)(id): I'm knocked out and out of mana so no more battles for me..."**
  * This Mage is not in a DemonHideout print **"(display_code)(id): I can only battle in a DemonHideout!"**
- This Mage does not have enough gold pieces print **"(display_code)(id): Not enough money for battles"**
- The current_hideout is done print **"(display_code)(id): Cannot battle! This DemonHideout has no more mages to battle!"**
- If this Mage can start training, set its `num_battles` to the requested battles and update the remaining battles in the hideout . This will be used when its `Update()` function is called.

- `void StartRecoveringMana(unsigned int num_crystals);`
  - Tells the Mages to start recovering at a ManaSpire.
  - Sets the state to `RECOVERING_MANA` and prints the message **"(display_code)(id): Started recovering (num_crystals) crystals at Mana Spire (current_spire_id)"** unless the following conditions are true:
    * This Mage does not have enough gold pieces print **"(display_code)(id): Not enough money to recover mana."**
    * The Mana Spire does not have at least one crystal remaining otherwise print **"(display_code)(id): Cannot recover! No crystal remaining in this Mana Spire"**
    * This Mage is not in a Mana Spire otherwise print **"(display_code)(id): I can only recover mana at a Mana Spire!"**
  - If this Mage can start recovering mana, set its `crystals_to_buy` to the minimum of the requested crystals and update the remaining crystals in the spire. This will be used when its `Update()` function is called.
  - Five mana is recovered for each crystal purchased. **(Piazza updates may change this as we tweak the game)**

- `void Stop();`
  - Tells this Mage to stop doing whatever it was doing.
  - Sets the state to `STOPPED`.
  - Prints **"(display_code)(id): Stopping.."**

- `bool IsKnockedOut();`

  - Returns `true` if mana is 0

- `bool ShouldBeVisible();`
  - Returns `true` if this mage is **not** knocked out

- `void showstatus();`
  - Prints **"(name) status: "**
  - Call `GameObject::ShowStatus()`
  - Print state specific status information. Refer to **How Mage Objects Behave** for complete details

- `bool Update()`
  - Check "How Mage objects behave part"

Protected members:

- `bool UpdateLocation();`
  - Updates the object's location while it is moving (See "How the Mage Moves" for details).
  - Prints **"(display_code)(id): I'm there!"** if a Mage has arrived at its destination.
  - **Prints "(display_code)(id): step..."** otherwise.

- `void SetupDestination(Point2D dest);`
  - Sets up the object to start moving to `dest`. (See "How the Mage Moves")

Private members:

- `double speed;`
  - The speed this object travels, expressed as distance per update time unit.

- `bool is_at_spire;`
  - Set to true of the Mage is in a ManaSpire
  - Initial value should be `false`.

- `bool is_in_hideout;`
  - Set to true if this Mage is in a DemonHideout.
  - Initial value should be false.

- `unsigned int mana;`

  - Amount of mana a Mage has
  - Initial value should be 20. **(Piazza updates may change this as we tweak the game)**

- `unsigned int experience;`
  - The amount of experience points this Mage has.
  - Initial value should be 0.

- `double gold_pieces;`
  - The total amount of gold pieces this Mage holds.
  - Initial value should be 0.

- `unsigned int battles_to_buy;`
  - Stores the number of battles to buy when in a DemonHideout
  - Initial value should be 0.

- `unsigned int crystals_to_buy;`
  - Stores the number of crystals to buy when in a ManaSpire
  - Initial value should be 0.

- `string name;`
  - The name of this Mage.

- `ManaSpire* currrent_spire`
  - A pointer to the current ManaSpire.
  - Initial value should be 0 (`NULL`).

- `DemonHideout* current_hideout`
  - A pointer to the current DemonHideout.
  - Initial value should be 0 (`NULL`).

- `Point2D destination;`
  - This object's current destination coordinates in the real plane.
  - Point2D's default constructor will initialize this to `(0.0, 0.0)`.

- `Vector2D delta;`
  - Contains the x and y amounts that the object will move on each time unit.
  - See "How Mage Moves" for more information.

Non-members (static):

- `double GetRandomAmountOfGP();`
  - Returns a random number between 0.0 and 2.0 inclusive. **(Piazza updates may change this as we tweak the game)**

How Mage Objects Move

The main function of the program accepts commands from the user. Simulated time is stopped while the user enters commands. The user can command individual objects to move to specified destination coordinates. When the user tells the program to "go", one step of simulated time then happens, and the program calls the update function on every object. The program then pauses to let the user enter more commands, and when the user commands "go" again, another step of simulated time happens, and every object is updated again. So each "go" command corresponds to one "tick" of the clock, one step of the simulated time. The "run" command conveniently makes the program run until an important event happens (to be defined).

An object is commanded to move by calling its `StartMoving()` function (inside the Mage class) and supplying the destination. The `StartMoving()` function does the following: Call the `SetupDestination` function to save the destination and calculate the delta value. Then set the object in the moving state. The delta value contains the amount that the object's x and y coordinates will change on each update step. We calculate it once and then apply it to each step. This is the purpose of the overloaded operators for `Point2D` and `Vector2D`. To calculate the value of delta, use:

```
    delta = (destination - location) * (speed / GetDistanceBetween(
       destination, location))
```

In other words, the object will move in a straight line, moving a distance equal to its speed on each step. The change in the x and y values of the location on each step are thus proportional to the ratio of the speed to the distance. So the `SetupDistance()` function calculates the delta value to be used in the updating steps.

On each step of simulated time, the main routine will call each object's Update() function. The update function for Mage does a variety of things, but if the object is moving, it calls the `UpdateLocation()` function. This function first checks to see if the object is within one step of its destination (see below). If it is, `UpdateLocation()` sets the object's location to the destination, prints an "arrived" message, and then returns true to indicate that the object arrived. If the object is not within a step of destination, `UpdateLocation()` adds the delta to the location, prints a "moved" messaged, and returns false to indicate that the object has not yet arrived. Thus the object will take a "speed-sized" step on each update "tick" until it gets within one step of the destination, and then on the last step, goes exactly to the destination.

Finally, notice that the user can command all of the Mage objects to move to a destination, and then tell the program to "go", and all of the objects will start moving, and each will stop when it arrives at its destination. The objects are responsible for themselves!

An object is within a step of the destination if the absolute value of both the x and y components of `fabs(destination - location)` *are less than or equal to the x and y components of delta (use the*

`fabs()` function in `math.h` library). By checking our distance with very simple computations using the delta value, we don't have to calculate the remaining `cart_distance` and compare it to the speed using a slow square root function on every update step.

How Mage Objects Behave

The behavior of the Mage class is programmed using an approach called a "state machine". A state machine is a system that can be in one of several states and behaves depending on what state it is in. It can either stay in the same state or change to a different state and behave differently. The neat feature of this approach is that you can easily specify a complicated behavior pattern by simply listing the possible states, and then with each state, describe the input/output behavior of the machine and whether it will change state (See http://en.wikipedia.org/wiki/Finite-state_machine for detail).

In the Mage class, the `Update()` function will do something different depending on the state of the Mage object. The state is represented with an enumerated type that contains a number code for the particular state. A good way to program this is to use a switch statement that switches on the state variable and has a case for each possible state. In each case, perform the appropriate action for the state, and if needed, change the state by setting the state variable to a different value. Then the next time the `Update()` function is called, the Mage will do the appropriate thing for the current state. Thus, the `Update()` function contains nothing but a big switch statement.

Here are the states of Mage, and what the `Update()` and `ShowStatus()` functions do for each state. Generally, the `Update()` function should return **`true`** whenever the state is changed and return **`false`** if it stays in the same state:

- `STOPPED`
  - The Mage does nothing and stays in this state.
  - `ShowStatus()` prints **" stopped"**
  - `Update()` should return **`false`**
- `MOVING`
  - `ShowStatus()` prints **" moving at a speed of (speed) to destination <X, Y> at each step of (delta)."**
  - `Update()` should
    * Call `UpdateLocation()` to take a step;
    * if the object has arrived, set the state to `STOPPED` and return **`true`**
    * Otherwise, stay in the `MOVING` state.
- `MOVING_TO_HIDEOUT`
  - `ShowStatus()` prints **" heading to DemonHideout (current_hideout id) at a speed of (speed) at each step of (delta)"**
  - `Update()` should
    * Call `UpdateLocation()`.

    * If it has arrived, set the state to `IN_HIDEOUT`, and return **`true`**
    * Otherwise stay in the `MOVING` state.

- `MOVING_TO_SPIRE`
  - `ShowStatus()` prints **" heading to Mana Spire (current_spire id) at a speed of (speed) at each step of (delta)"**
  - `Update()` should
    * Call `UpdateLocation()`. If it has arrived, set the state to `AT_SPIRE` and return **`true`**
    * Otherwise stay in the current state.

- `IN_HIDEOUT`
  - `ShowStatus()` prints **" inside Demon Hideout (current_hideout id)"**
  - `Update()` should return **`false`**

- `AT_SPIRE`
  - `ShowStatus()` prints **" inside Mana Spire (current_Spire id)"**
  - `Update()` should return **`false`**

- `BATTLING_IN_HIDEOUT`
  - `ShowStatus()` prints **" battling in DemonHideout (current_hideout id)."**
  - `Update()` should:
    * reduce Mage mana based on total mana cost for the current hideout request
    * reduce the amount of gold pieces based on the dollar cost for the current hideout request
    * increase Mage experience based on experience gain for the current hideout request; this should be calculated using `TrainMage()`
    * print **"** (name) completed (battles_to_buy) battle(s)! **"**
    * print **"** (name) gained (experience gained) experience! **"**
    * Set state to `IN_HIDEOUT` and return **`true`**

- `RECOVERING_MANA`
  - `ShowStatus()` prints **" recovering mana in Mana Spire (current_spire id)"**
  - `Update()` should
    * Increase Mana; mana should be calculated by `StartRecoveringMana()`
    * Reduce gold pieces by the total cost of crystals for the current ManaSpire
    * Prints **"** (name) recovered (mana recovered) mana! **"**
    * Prints **"** (name) bought (crystals_recieved) crystal(s)! **"**
    * Set state to `AT_SPIRE` and return **`true`**

In all states print the following:

- **"Mana: (mana)"**
- **"Gold Pieces: (gold_pieces)"**
- **"Experience: (experience)"**

Thus, if the Mage is commanded by `StartMoving` to a destination, it goes into the moving state, starts moving, and then stops when it arrives and does nothing until commanded again. **For each "speed-sized" step the Mage should decrease their mana by 1 (battling roaming demons) and increase their gp count by a random amount. This is `true` for any time the Mage moves.**

If the Mage is supposed to battle the following happens: the `StartMovingToHideout()` function sets the `current_hideout` pointer member variable and the calls `SetupDestination()` to target the `current_hideout` and sets the state to `MOVING_TO_HIDEOUT`. Once at a DemonHideout, Mage's state becomes `IN_HIDEOUT` and they can now battle to gain experience. Use the `StartBattling()` function to command a Mage to complete a specified number of battles. If the Mage does not have enough mana or gold pieces, their state should remain the same. If, however, a Mage can afford the battles set the state to `BATTLING_IN_HIDEOUT`. After one update tick, experience, mana and gold pieces are updated to reflect completing a training session. Finally, its state is then set to `IN_HIDEOUT`
.

Eventually, your Mage will need to recover lost mana. Use `StartMovingToSpire()` to command your Mage to move to a ManaSpire. `StartMovingToSpire()` sets the state to `MOVING_TO_SPIRE` and sets the `current_spire` member pointer variable to the requested Spire. Once a Mage reaches a spire, its state is set to `AT_SPIRE`. Mana can now be recovered through the `StartRecoveringMana()` function which sets the Mage's state to `RECOVERING_MANA`. If the Mage can afford the total amount of crystal requested, after one update tick, the Mage's mana is updated and its state is set to `AT_SPIRE`. Otherwise, the Mage remains in its current state.

Note

- If a Mage runs out of mana it should not be able to move!
  - When a Mage runs out of mana `Update()` should print **"(name) is out of mana and can't move"**
- The state should then be set to `KNOCKED_OUT`
- When a Mages leaves or enters a building it should call the building's `RemoveOneMage()` and `AddOneMage()` function respectively
- A Mage can only recover mana when its state is `AT_SPIRE`. Likewise, a Mage can only battle if its state is `IN_HIDEOUT`.
- If a Mage requests more crystals or battles than are available, they should get the available amount.

CHECKPOINT III

Iteratively develop the Mage class. **Start implementing the class by only writing the constructors and a partial form of the `ShowStatus()` function; leave everything else out.** To test the correct behavior, write another simple `TestCheckpoint3.cpp` file. In the main function, create a Mage object, and call its `ShowStatus()` function. It should display the correct initial state of the objects, so you can verify if both the constructors and the `ShowStatus()` function work properly. Only then start implementing the `StartMoving()`, `SetupDestination()`, and `UpdateLocation()` functions, and `STOPPED` and `MOVING` parts of the update and `ShowStatus()` functions.

Change your trivial main function to call the `StartMoving()` function on your one Mage object, showing its status, calling its `Update()` function, and showing its status again - it should be in a different location! Check that amount moved on the step is correct. Put in a few more calls of update and see if the Mage stops like it should. With the help of `ManaSpire` and `DemonHideout` objects, you are able to battle with your Mage and recover its mana after it becomes tired.

DO NOT MOVE ON UNTIL YOU ARE 100% SURE YOU UNDERSTAND THIS CHECKPOINT

If you do not make sure that the Mage class works fully, the rest of your game will NOT work. Use this time to also start to modify the provided makefile to also work for testing Checkpoint 3. DON'T RUSH THROUGH THIS.